Informe de arquitectura

PharmaSoft · Lenguaje de Programación II · Sesión 7 · Actividad autónoma

B1. ¿Cómo está organizado el proyecto?

El frontend separa las partes generales de las funciones de negocio. Así, cada módulo tiene su propio código y el menú y la estructura visual se comparten.

Carpeta	Para qué sirve
core	Configuración, menú, tipos comunes y manejo de errores.
features/categorias	Pantallas y servicios para administrar categorías.
features/clientes	Modelos, servicio, páginas y rutas de clientes.
layout	Encabezado, menú lateral y estructura que rodea las páginas.
shared/pages	Página que se muestra cuando la ruta no existe.

B2. Rutas principales

Las rutas de Clientes y Categorías se cargan cuando se necesitan. Ambas aparecen dentro de la misma estructura visual.

Dirección	Pantalla
/inicio	Inicio
/categorias · /categorias/nuevo · /categorias/:id/editar	Lista o formulario de categorías
/clientes · /clientes/nuevo · /clientes/:id/editar	Lista o formulario de clientes
Cualquier ruta desconocida	Página 404

Archivos revisados: app.routes.ts, clientes.routes.ts y categorias.routes.ts.

B3. Responsabilidades

En cada módulo, la pantalla se ocupa de la interacción y el servicio se comunica con el backend. El layout mantiene la navegación común.

Parte	Responsabilidad
ClienteList	Muestra clientes; controla búsqueda, orden, páginas y estados de carga o error.
ClienteForm	Valida datos y sirve para crear o editar un cliente.
ClienteService	Envía las solicitudes HTTP de clientes al backend.
CategoriaList / CategoriaForm	Muestran y editan categorías siguiendo el mismo patrón.
MainLayout, Header y Sidebar	Mantienen la estructura, los accesos y el menú lateral.

La lista no llama directamente a HttpClient. Pide los datos al servicio para mantener separada la pantalla de la comunicación con el backend.

Archivos revisados: cliente-list.ts, cliente-form.ts, cliente-service.ts, main-layout y componentes de navegación.

B4. ¿Qué pasa al guardar un cliente?

El formulario valida la información y el servicio la envía al backend. Si se guarda correctamente, se vuelve a la lista.

1. Formulario	2. Servicio	3. Backend
Valida los campos y prepara los datos.	Envía POST /api/v1/clientes.	Revisa los datos y los guarda.

Si todo sale bien: el backend responde 201, la aplicación abre la lista y esta vuelve a pedir los clientes. Si el cliente ya existe, responde 409 y el formulario muestra el error para corregirlo.

> Los campos opcionales vacíos se envían como null. El formulario conserva lo escrito cuando el backend devuelve un error.

Archivos revisados: cliente-form.ts, cliente-service.ts, cliente-list.ts y http-error.ts.

B5. Preguntas de revisión

1. ¿Qué archivos existentes se tocaron para agregar Clientes?

En el frontend se agregó la ruta en app.routes.ts y el enlace en core/config/menu.ts. Las demás piezas se ubicaron dentro de features/clientes. También se ajustó el backend porque esta copia no tenía la paginación y la baja lógica que usa la actividad.

2. ¿Por qué la lista no usa HttpClient directamente?

Porque ClienteService concentra las llamadas al backend. Si cambia la dirección de la API o cómo se envían los parámetros, el cambio queda en el servicio y no repartido por la pantalla.

3. ¿Qué aporta loadChildren?

Permite cargar las rutas del módulo cuando se entra en él. Esto ayuda a separar Clientes y Categorías y reduce lo que se carga al inicio.

4. ¿Por qué el encabezado y el menú siguen al cambiar de página?

Porque están dentro de MainLayout, que contiene el espacio donde se muestra la página seleccionada. Al cambiar entre módulos, cambia la página central y se mantiene el marco común.

Archivos revisados: app.routes.ts, core/config/menu.ts, main-layout y rutas de los módulos.

B5. Preguntas de revisión (continuación)

5. ¿Dónde se podría controlar qué enlaces ve cada usuario?

El menú se define en core/config/menu.ts, así que ese es un lugar práctico para filtrar enlaces. Ocultar un enlace no protege por sí solo la ruta ni los datos: eso debe validarse también en el backend.

6. ¿Qué se podría reutilizar entre Categorías y Clientes?

Se parecen en sus listas, formularios y estados de carga. Si se agregan más módulos, se podrían compartir controles sencillos, como el paginador o los mensajes de error. Las reglas y los datos de cada módulo deben seguir separados.

7. ¿La búsqueda revisa todos los clientes?

No. Actualmente filtra los clientes de la página que ya recibió. Para buscar en todos, el backend tendría que aceptar el texto de búsqueda y combinarlo con el orden y la paginación.

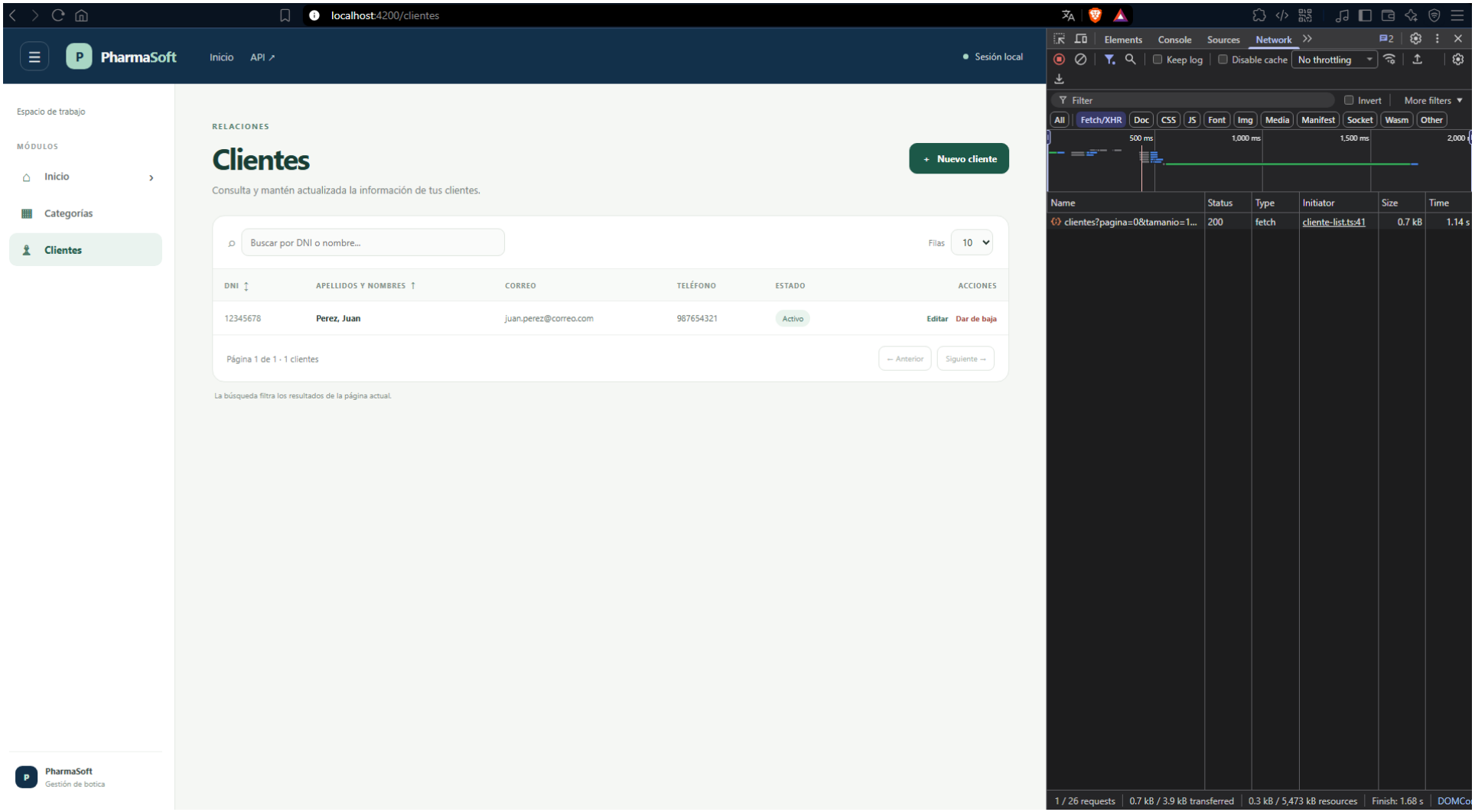
Comprobación

Se compiló el frontend y se realizaron las pruebas manuales mostradas en el anexo. Las capturas incluyen las respuestas del backend en la pestaña Network.

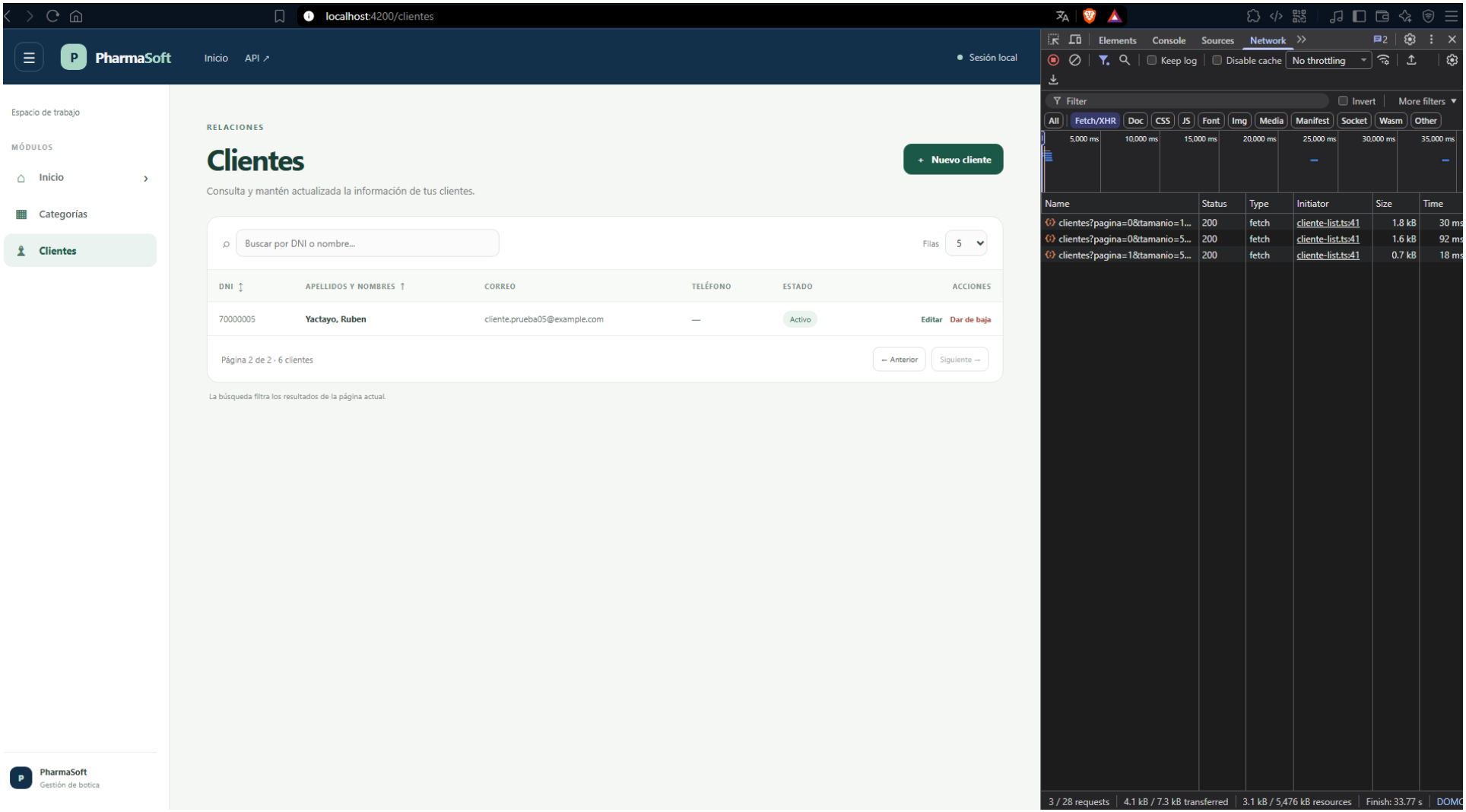
Nota: el informe resume la implementación y las respuestas de la actividad. El anexo contiene las capturas solicitadas como evidencia.

Archivos revisados: ClienteList, ClienteForm, ClienteService y sus equivalentes del módulo Categorías.

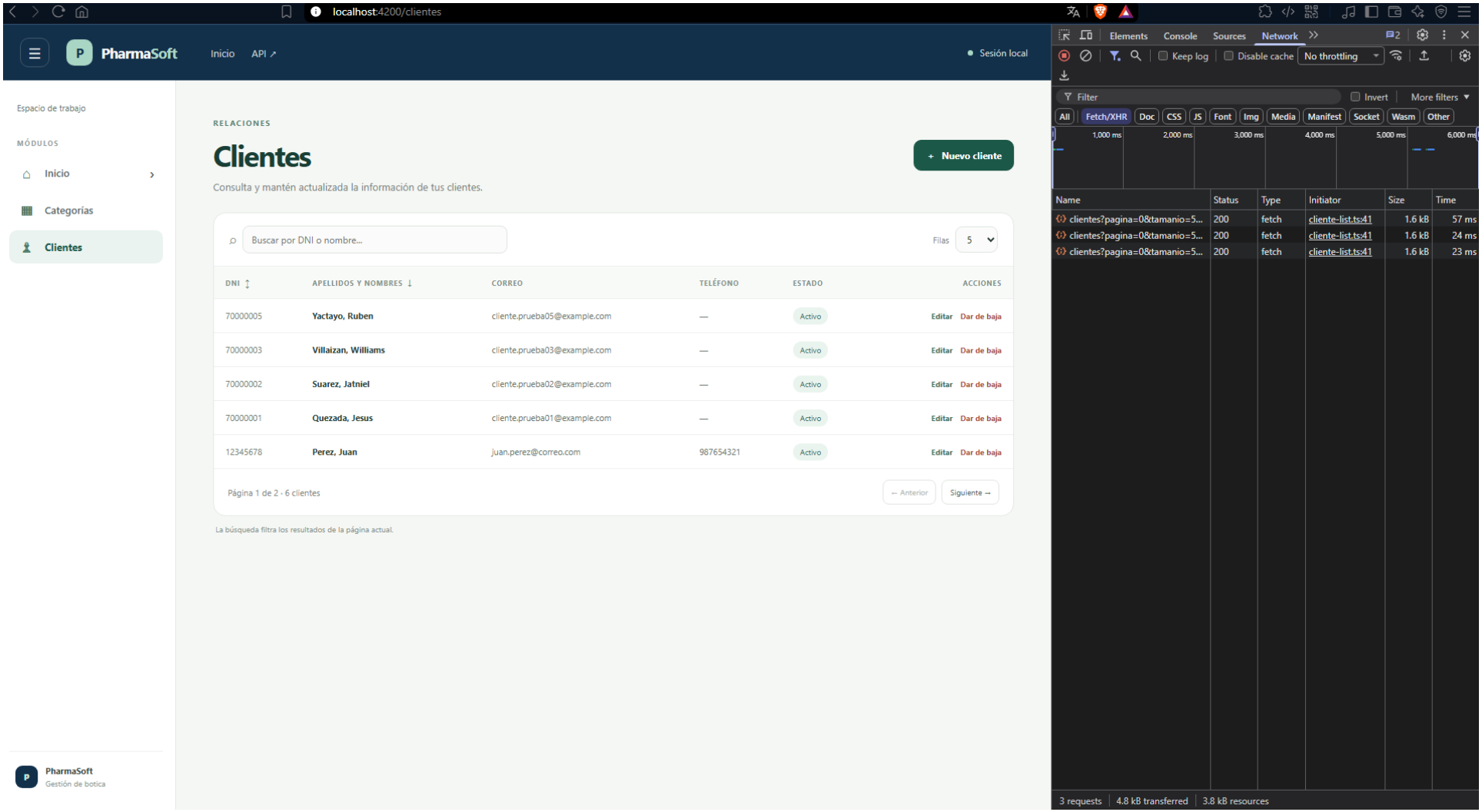
A7.1 - Navegacion desde el sidebar; listado inicial y GET 200.



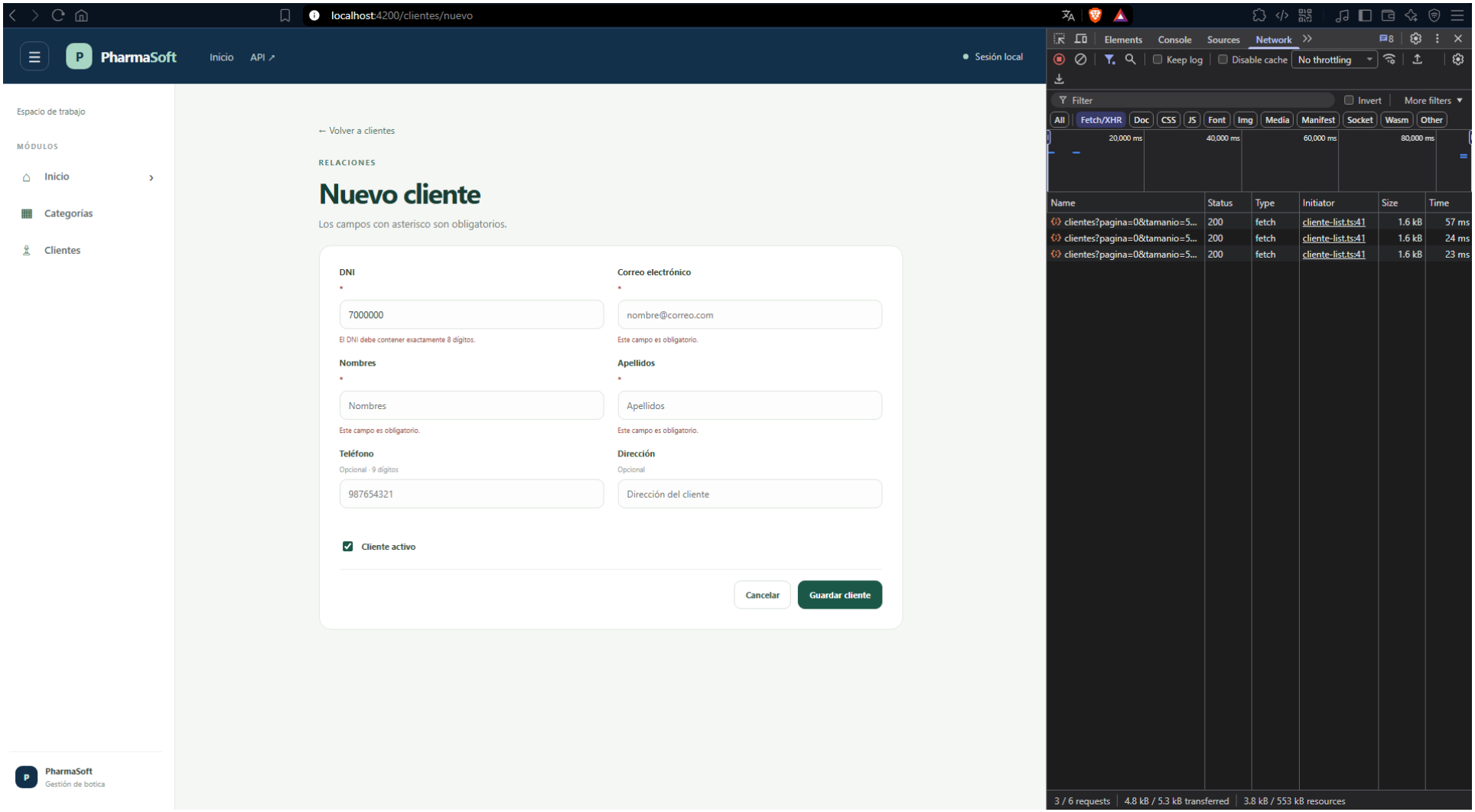
A7.2 - Tamano de pagina 5 y navegacion a pagina 2.



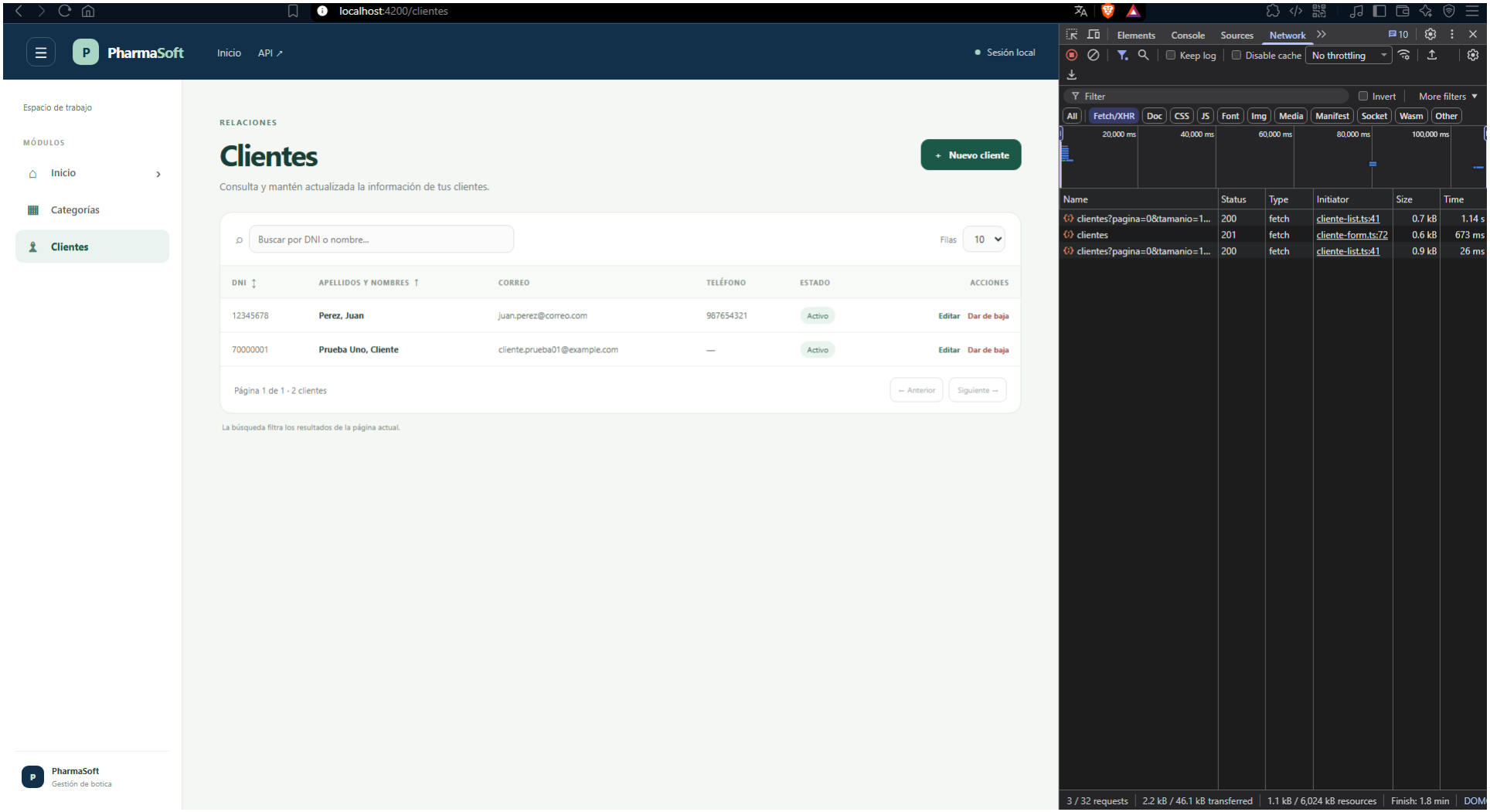
A7.3 - Orden por apellidos alternado; lista en orden descendente.



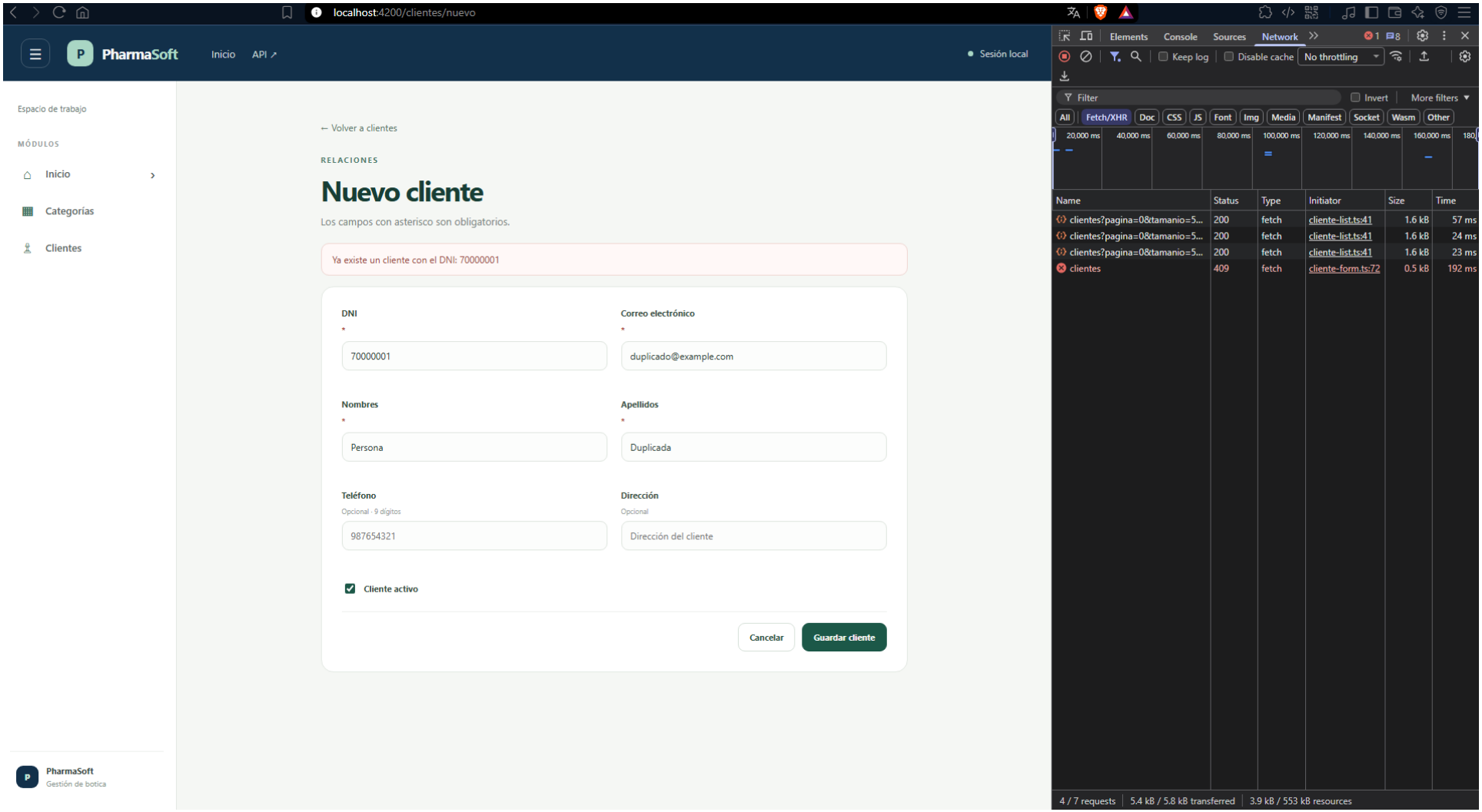
A7.4 - Validacion de DNI de siete digitos sin POST.



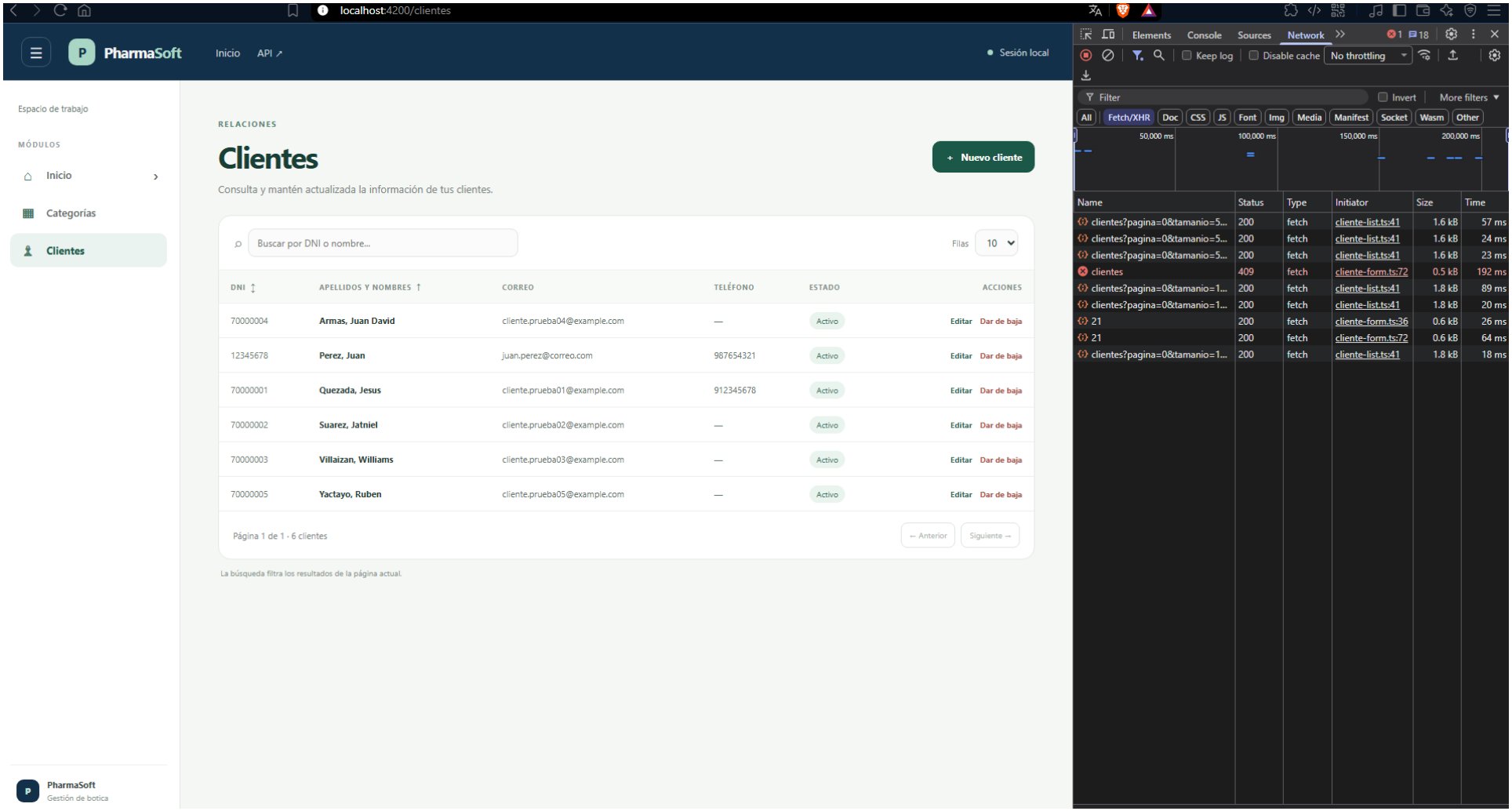
A7.5 - Alta valida sin telefono; POST 201 y telefono mostrado como raya.



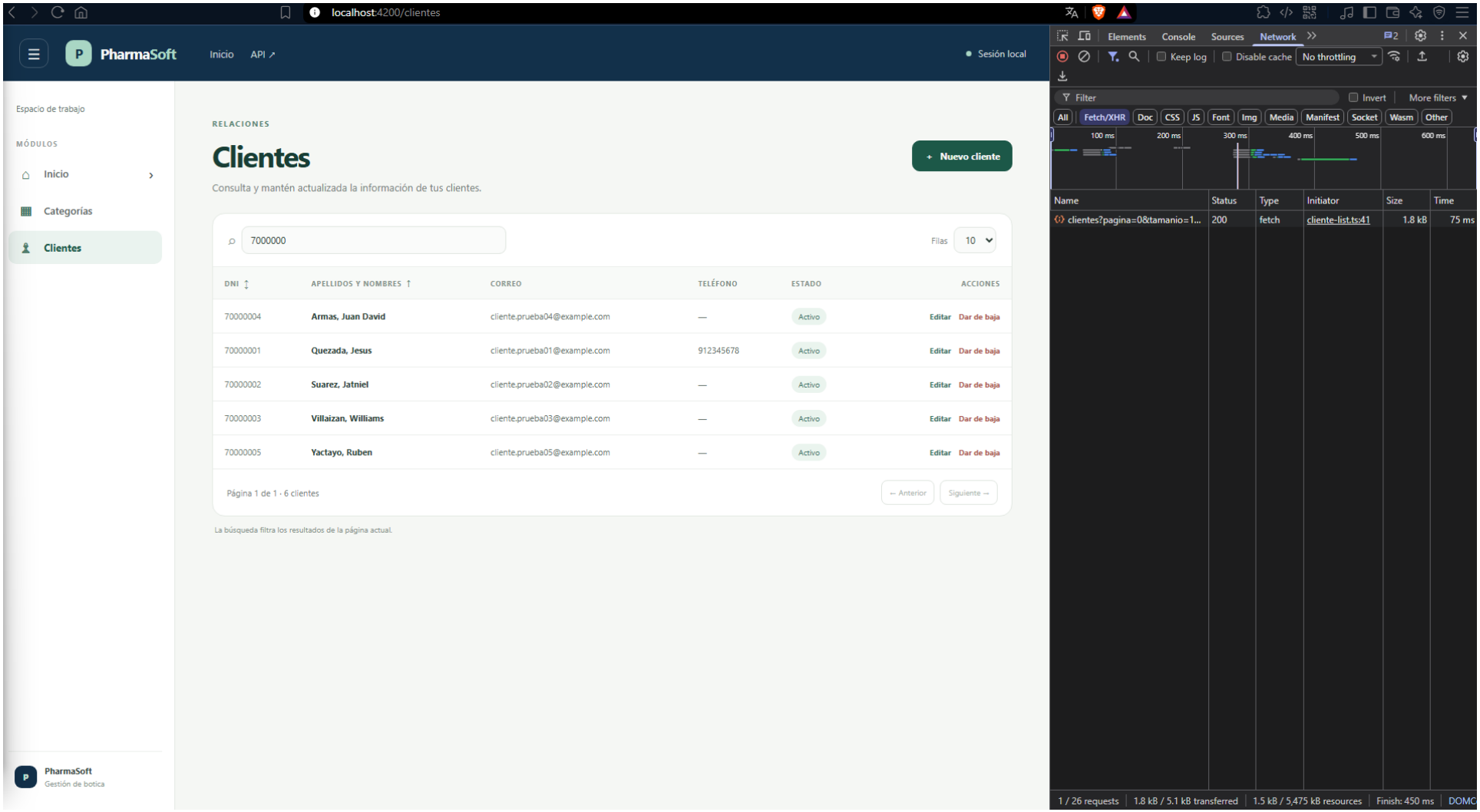
A7.6 - DNI duplicado; mensaje del backend y POST 409.



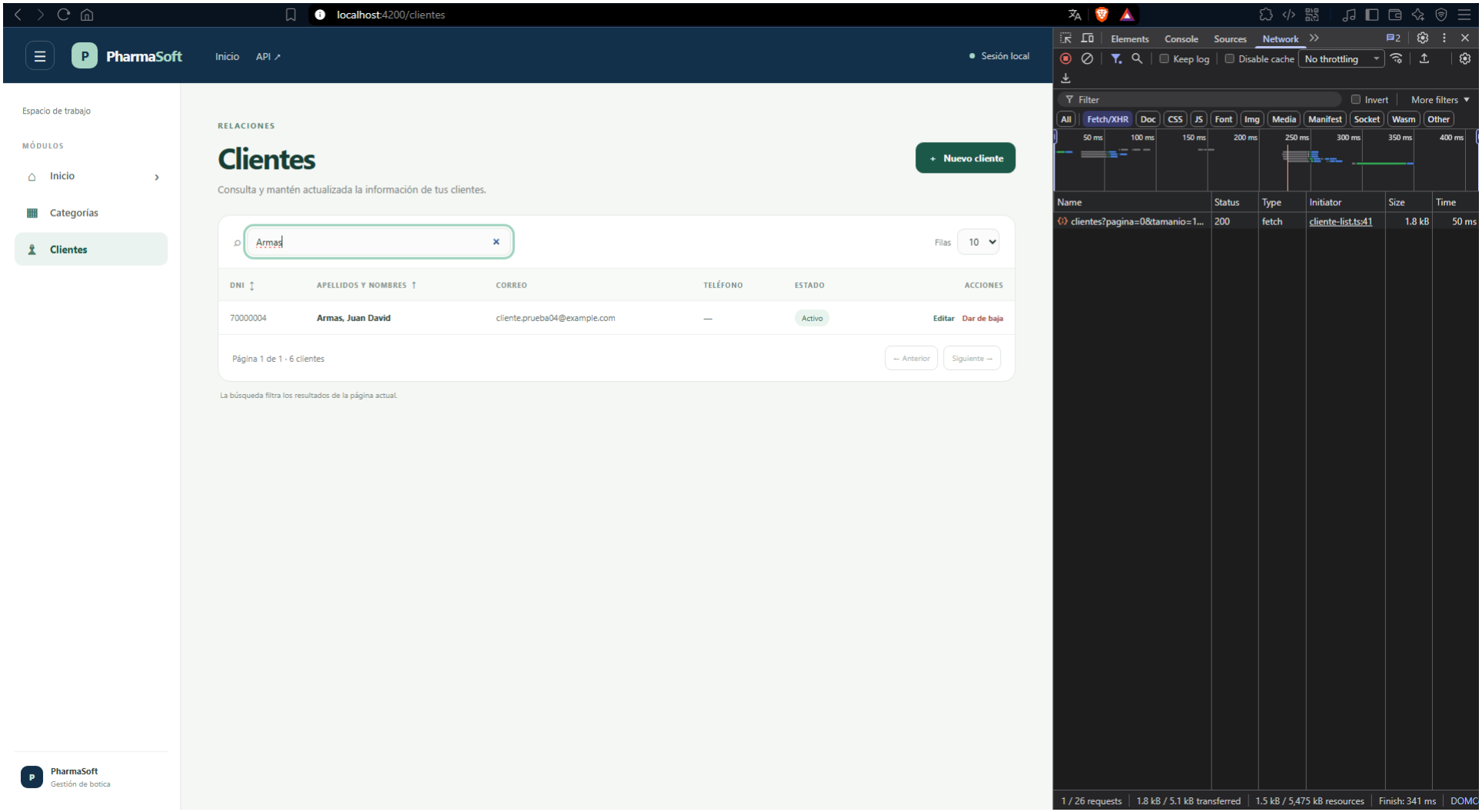
A7.7 - Edicion de telefono; fila actualizada y PUT 200.



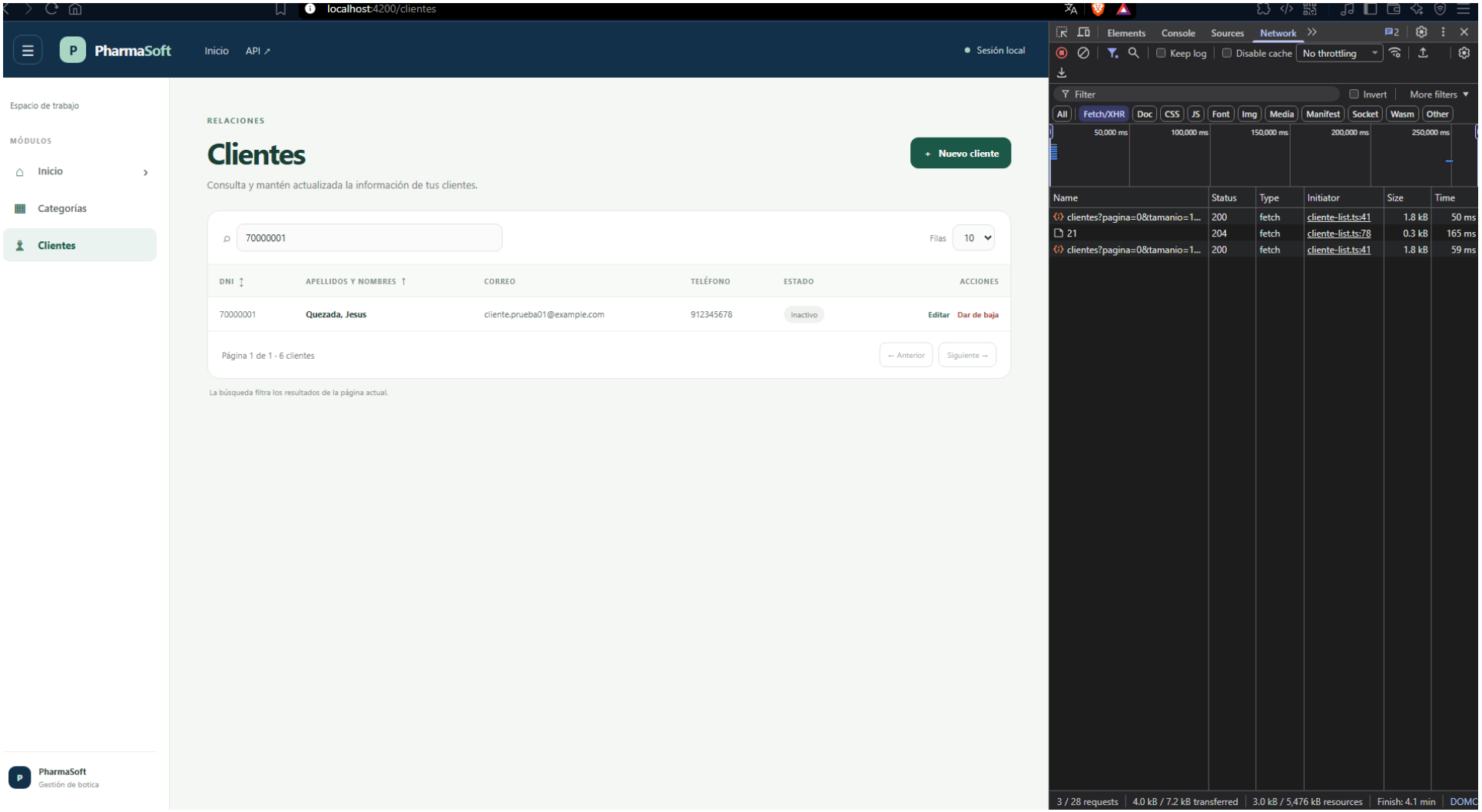
A7.8a - Busqueda por fragmento del DNI dentro de la pagina actual.



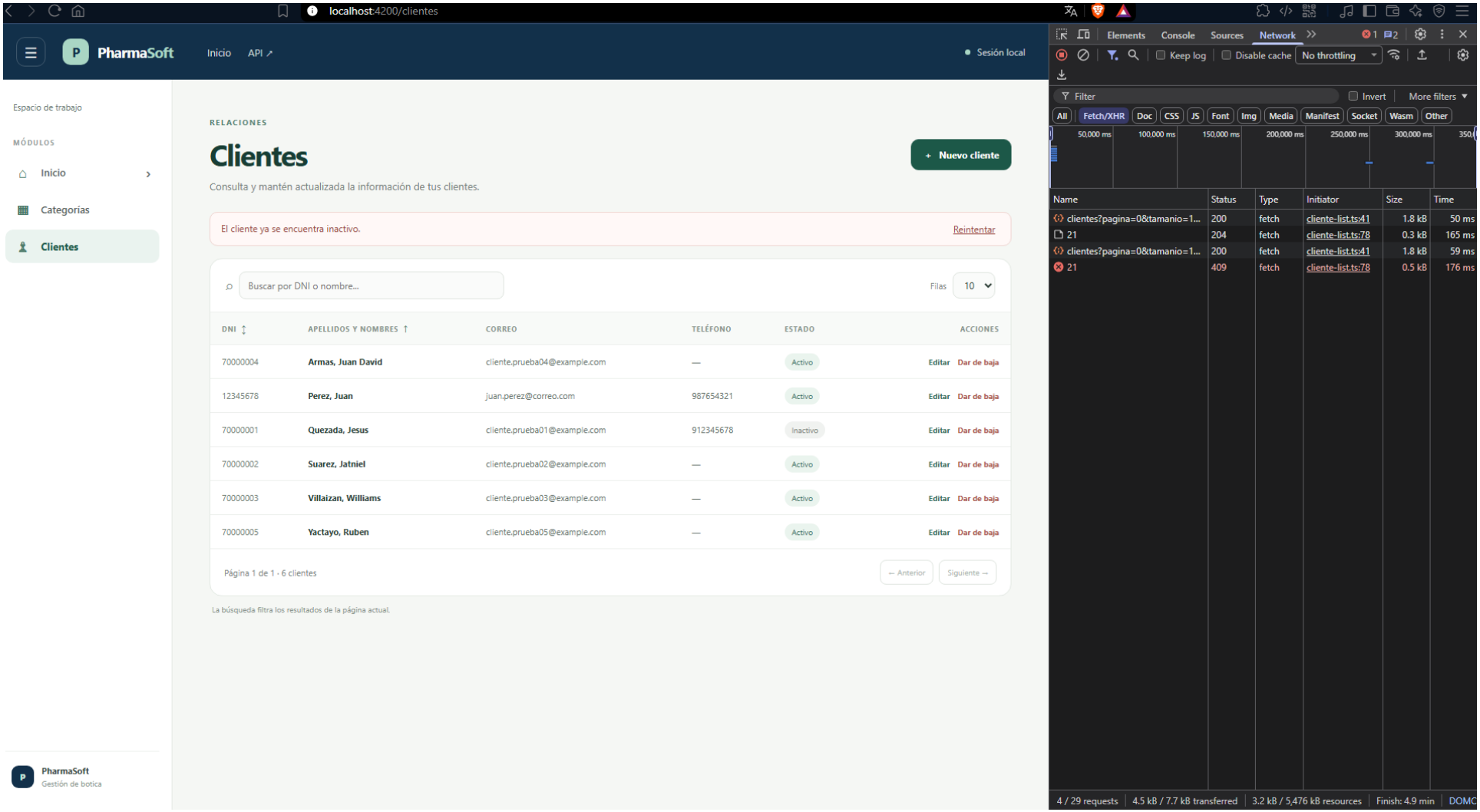
A7.8b - Busqueda por apellido; resultado Armas sin peticion nueva por el filtro.



A7.9 - Baja logica confirmada; cliente Inactivo y DELETE 204.



A7.10 - Segundo intento de baja; mensaje y DELETE 409.



A7.11 - Regreso a Categorias; catalogo cargado y GET 200.

